

Design Rationale

REQ 2: Be wary of dung (beetle)

A. GoldenBeetle Implementation

Design 1: Create GoldenBeetle class extending Creature abstract class and implement Eatable and Producible interfaces

Pros	Cons
Promotes code reusability	Requires maintenance for extra class and interfaces
Clear modularisation of responsibilities using interfaces and abstract classes	Results in complex program if every functionality needs abstract classes and interfaces
Enhances code readability as the abstract class and interfaces define a clear contract of what is expected for GoldenBeetle class	

Design 2: Implement all logic directly inside the GoldenBeetle class

Pros	Cons
Enhances code maintainability as no additional classes or interfaces need to be maintained	GoldenBeetle class takes on multiple responsibilities
	Existing code need modification if new behaviours are added

B. Following Behaviour

Design 1: Directly implement `getAction()` and `distance()` methods in GoldenBeetle class

Pros	Cons
Enhances code maintainability as no additional class need to be used	Class is responsible for both its behaviour and following logic
Simpler implementation if only GoldenBeetle can follow an actor	Duplicated code will be implemented across every actor with the ability to follow a followable actor

Design 2: Create a FollowBehaviour class

Pros	Cons
Promotes code reusability as other actors can easily reuse this class	Requires maintenance of an extra class
Allows runtime modification	Connascence of Meaning – Modification needed if the meaning and implementation of the behaviour change
GoldenBeetle is loosely couple to the following logic by simply adding FollowBehaviour to its map	

C. Restore Player's Stamina after Consuming GoldenEgg

Design 1: Parse the parent of the Egg when creating a new Egg, check the parent's type using 'instanceof' and restore player's stamina directly if the identified parent is a GoldenBeetle in the eat() method in Egg class

Pros	Cons
Simpler and direct implementation	Modification needed when a new type of egg is added with different consumption effect
Enhances code maintainability as no additional classes or interfaces need to be maintained	Violates Liskov Substitution Principle - Requires checking of type of creature using 'instanceof'

Design 2: Create a new class RestoreStaminaEffect and store it in Egg's consumption effects to apply the effects when the egg is eaten in the eat() method in Egg class

Pros	Cons
Enhance code extensibility as any new effect can be simply applied by adding the class in the egg's list of consumption effects	Requires maintenance for an extra class
Enhance code reusability as other classes that have the same effect can reuse this class	

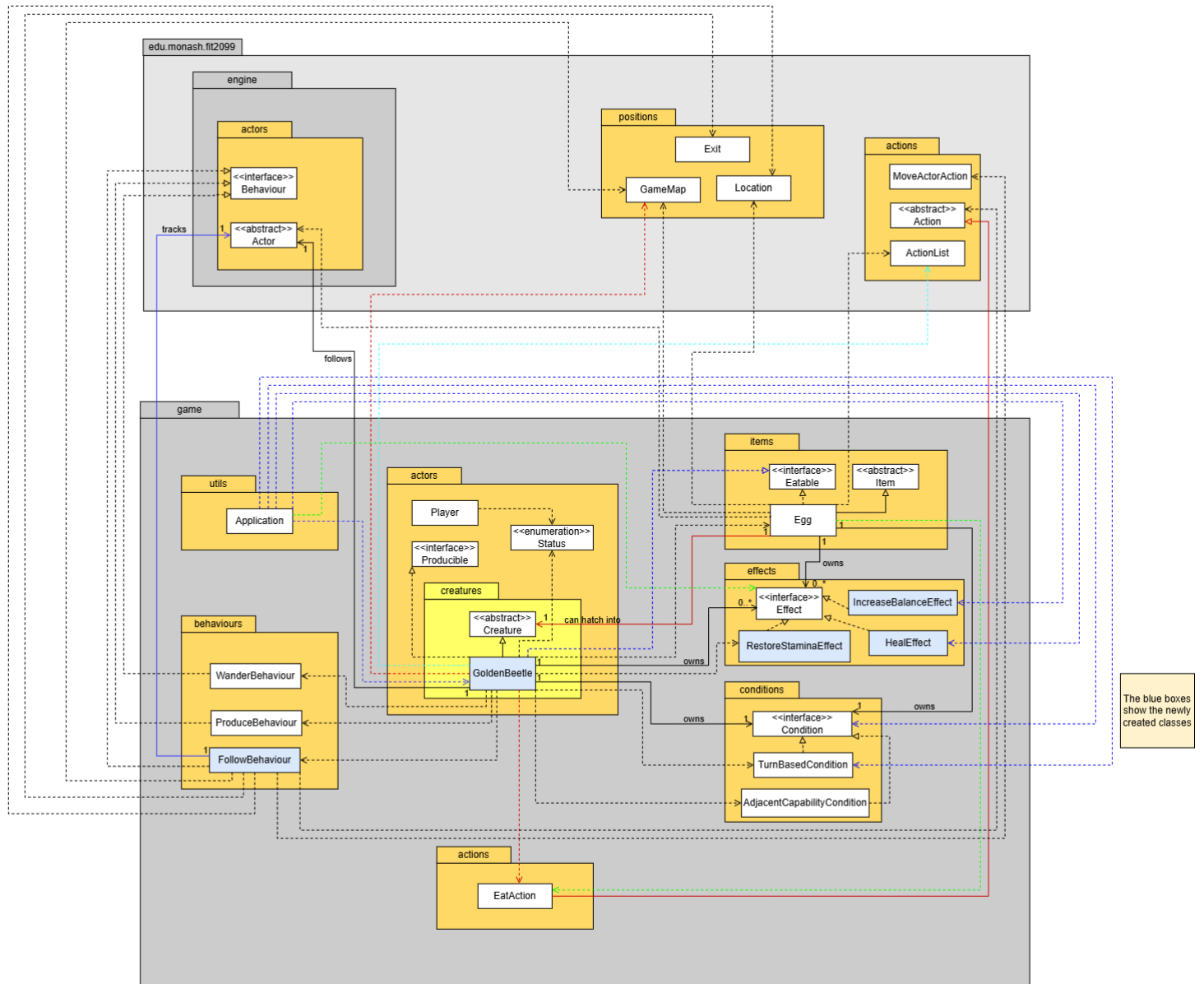
D. Hatching Condition of GoldenEgg

Design 1: Use 'instanceof' and downcasting to check the type of ground that GoldenEgg can hatch on in the shouldHatch() method of Egg class

Pros	Cons
Simpler and direct implementation	Modification needed when a new cursed entity is added
	Violates Liskov Substitution Principle - Requires checking of type of ground using 'instanceof'

Design 2: Create a new class AdjacentCapabilityCondition that utilises Status enum class to check if GoldenEgg can be hatched (located near CURSED entities)

Pros	Cons
Enhance code extensibility as the status can be added to future cursed entity that allow GoldenEgg to be hatched nearby it	Additional status logic implementation in the ground classes needed
Enhance code readability	



The diagram above shows the final design of REQ2, which utilizes Design 1 for GoldenBeetle implementation, Design 2 for following behaviour, restoration of stamina after consuming a GoldenEgg and hatching condition check of GoldenEgg.

The GoldenBeetle class extends the abstract Creature class. The new concrete class inherits the methods and attributes from its existing abstract class. We abstract these identities to avoid repetition in code (DRY) as this class shares many same attributes and methods with the abstract class it extends from. The new creature created solely uses the constructor of its super class with different initialized values, enabling it to replace the usage of their subclass in code (Liskov Substitution Principle).

In the GoldenBeetle class, the allowableActions() is overridden to add an eat action and follow behaviour when it's nearby an Followable actor, giving GoldenBeetle class the responsibility of adding an eat action to the options, reducing dependencies of other classes on this class but segregating this responsibility (Single Responsibility Principle). Besides, GoldenBeetle class relies on abstraction rather than concrete implementation, adhering Dependency Inversion Principle. Moreover, using different interfaces to define different traits (i.e. can produce) and abilities (i.e. ability to spawn and be eaten) of the concrete GoldenBeetle class allows us to separate different responsibilities to different interfaces, resulting in better maintainability of the system (Single Responsibility Principle). This implementation allows the GoldenBeetle's traits to be separated by their own purposes (Interface Segregation Principle). Future actions and behaviours can simply be added or implemented by the GoldenBeetle class, without changing the existing classes, adhering to the Open Closed Principle.

Adding a new FollowBehaviour allows us to segregate the logic of tracking the movement of an actor and moving following the target from the GoldenBeetle class itself, rather than embedding the following logic directly in the GoldenBeetle class by implementing getAction() and distance(). Using this implementation, the GoldenBeetle class will be responsible for its own purpose only, where different responsibilities are distributed to different classes (Single Responsibility Principle). As this design adheres to the Open Closed Principle, there will be minimal impact on the existing class if any new requirement for the following behaviour arises (e.g. adding logic to only follow the target when it is within a specific range in FollowBehaviour class). The priority of behaviours is managed using an order-based TreeMap.

Creating a new class RestoreStaminaEffect enhances code reusability as this effect can be shared across different classes without duplicating the logic (DRY). We also avoided using 'instanceof' to check the parent of the egg and apply consumption effects based on the parent identified, adhering to the Liskov Substitution Principle. Moreover, this design adheres to the Open Closed Principle by allowing new consumption effects to be introduced and added (e.g. HealEffect class can be easily added to the list of consumption effects of the creature) without modifying the existing code.

Egg class will depend on Status enum class, which acts as an indicator to allow GoldenEgg to be hatched when it is nearby cursed entities. Using the dependency on the Status enum class, we avoided using downcasting to check for the types of the concrete classes that are fine to be hatched on, resulting in easier functionality extension without modifying existing code with additional if-else checks by allowing new ground to have CURSED status(Open-Closed Principle).

While connascence cannot be entirely eliminated, this final design has reduced the severity of connascence. Using interfaces and abstract classes in GoldenBeetle class, the GoldenBeetle class have implemented all required methods and adhered to the methods' signature (Connascence of Name). Besides, in the Egg class, the Egg depends on Condition, Creature and Effect classes (which are all abstract classes), making it extensible and minimising changes when those implementations evolve. For example, when new condition, creature or effect is introduced, no modification is needed for Egg class due to the use of polymorphism (Connascence of Type).